

GSFlow: Video Rendering Acceleration for 3D Gaussian Splatting via Geometry Pixel Flow

Supplementary Material

A. Appendix

A.1. Dataset Details

Our test dataset comprises a series of videos from both public sources and our own recordings, with detailed statistics provided in Table 5. To ensure reliable camera parameter extraction with COLMAP, some videos are sub-sampled at fixed intervals, resulting in varying frame rates across the dataset. Our method performs consistently well across these scenarios, handling both small inter-frame differences in high-FPS videos and larger displacements in low-FPS sequences, which demonstrates its strong robustness.

Table 5. Details of the Test Dataset

Scene	Images	Resolution	FPS
Tanks&Temples			
Meetingroom	1000	1920×1080	30
Barn	1000	1920×1080	30
Panther	800	1920×1080	15
Church	600	1920×1080	15
Custom Video Dataset			
Princess	1135	1316×1759	30
Small Dragon	926	1316×1759	30
Plant Whitepalm	533	1316×1759	30
Tiger	624	708×1263	15
Elephant	624	708×1263	15

A.2. Inpainting Network Architecture

Our proposed inpainting network adopts a UNet architecture. To accelerate inference, we optimize the network through two key approaches, including input downsampling and network compactness. We apply 4× input downsampling to significantly reduce computational load. The network structure begins with a convolution expanding channels to 64, followed by two downsampling convolutions increasing channels to 256 while reducing spatial dimensions by 4×. Two upsampling convolutions then recover feature dimensions, succeeded by two independent upsampling layers restoring the original resolution. A final convolution projects features to 3 output channels. The detailed structure is shown in Algorithm 2.

Algorithm 2: Inpainting Network Architecture

```
def __init__(self, n_channels=7, n_classes=3):
    super(UNet, self).__init__()
    self.n_channels = n_channels
    self.n_classes = n_classes

    self.inc = (DoubleConv(n_channels, 64))
    self.down1 = (Down(64, 128))
    self.down2 = (Down(128, 256))
    self.up1 = (Up(256, 128))
    self.up2 = (Up(128, 64))
    self.up3 = (Up_only(64, 16))
    self.up4 = (Up_only(16, 8))
    self.outc = (OutConv(8, n_classes))

def forward(self, x):
    x = interpolate(x, scale_factor=0.25)
    x1 = self.inc(x)
    x2 = self.down1(x1)
    x3 = self.down2(x2)
    x = self.up1(x3, x2)
    x = self.up2(x, x1)
    x = self.up3(x)
    x = self.up4(x)
    logits = self.outc(x)
    return logits
```

A.3. Customized CUDA Parallel Acceleration

We developed a custom CUDA kernel, detailed in Algorithm 3, to perform parallel nearest-neighbor inpainting on masked feature maps. The kernel employs a grid-stride loop to assign each pixel to a GPU thread. If a pixel is valid (mask=0.0), its value is preserved. For invalid pixels (mask=1.0), an efficient search for the nearest valid neighbor is executed. This search expands outwards in concentric square layers, systematically checking pixels on each boundary. Specifically, for a given search radius, the kernel first inspects all pixels along the top and bottom boundaries of the square, followed by the left and right boundaries. This structured, layer-by-layer inspection guarantees that the first valid pixel found is one of the closest in Manhattan distance. To maximize performance, the multi-level search is terminated instantly via a goto statement as soon as this first valid neighbor is located. The value of this neighbor is then used to fill the invalid pixel. This approach ensures that missing information is filled by the spatially closest available data point, preventing the introduction of interpolated artifacts and preserving sharp features.

Algorithm 3: Core CUDA Code of Filtering-based Inpainting Method

```

__global__ void interpolate_kernel(
    const float* projection,
    const float* mask,
    float* output,
    const int height,
    const int width)
{
    const int index = blockIdx.x * blockDim.x + threadIdx.x;
    const int stride = gridDim.x * blockDim.x;

    const int num_pixels_per_channel = height * width;
    for (int i = index; i < 3 * num_pixels_per_channel; i += stride) {
        if (mask[i] == 0.0f) {
            output[i] = projection[i];
            continue;
        }

        bool found = false;
        float found_val = 0.0f;
        const int c = i / num_pixels_per_channel;
        const int y = (i % num_pixels_per_channel) / width;
        const int x = i % width;
        const int max_radius = max(height, width);
        for (int radius = 1; radius < max_radius; ++radius) {
            // Search upper and lower boundaries
            for (int dx_offset = -radius; dx_offset <= radius; ++dx_offset) {
                int ny_top = y - radius;
                int nx_top = x + dx_offset;
                if (ny_top >= 0 && ny_top < height && nx_top >= 0 && nx_top < width) {
                    const int local_idx = c * num_pixels_per_channel + ny_top * width + nx_top;
                    if (mask[local_idx] == 0.0f) {
                        found_val = projection[local_idx];
                        found = true;
                        goto search_end;
                    }
                }

                int ny_bottom = y + radius;
                int nx_bottom = x + dx_offset;
                if (ny_bottom >= 0 && ny_bottom < height && nx_bottom >= 0 && nx_bottom < width) {
                    const int local_idx = c * num_pixels_per_channel + ny_bottom * width + nx_bottom;
                    if (mask[local_idx] == 0.0f) {
                        found_val = projection[local_idx];
                        found = true;
                        goto search_end;
                    }
                }
            }
            // Search left and right boundaries
            ...
        }

        search_end;;

        if (found) {
            output[i] = found_val;
        } else {
            output[i] = projection[i];
        }
    }
}

```

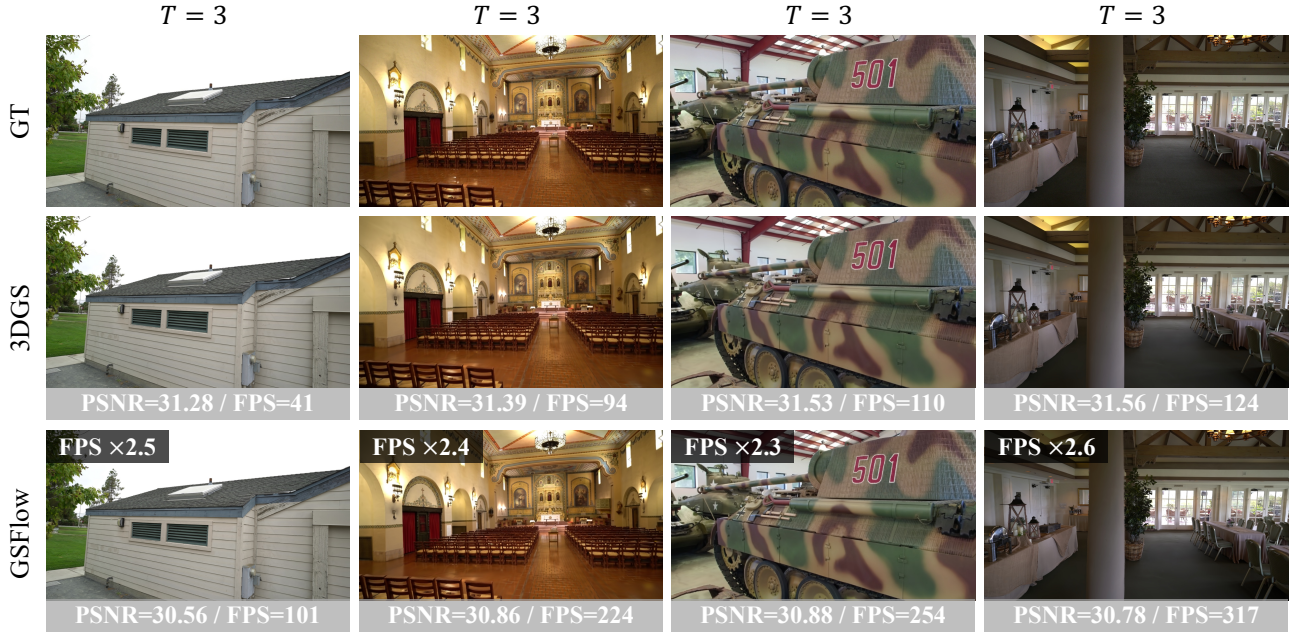


Figure 7. Rendering Comparisons under CUDA Environment.

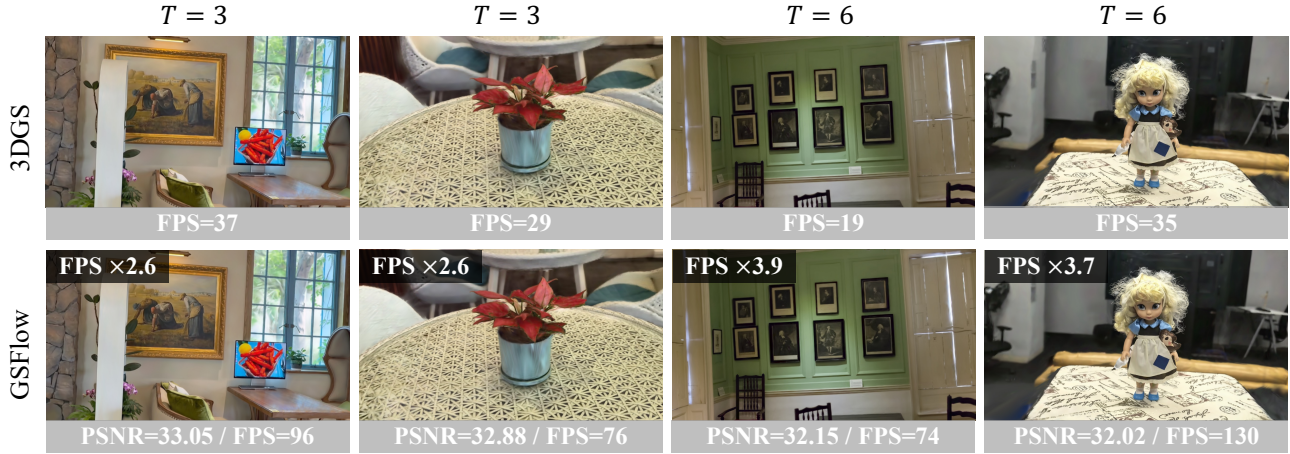


Figure 8. More Rendering Comparisons under Vulkan Implementation.

A.4. More Visualizations

Fig. 7 presents a visual comparison between our method and 3DGS under CUDA environment using a single RTX 4090 GPU. Since CUDA-based evaluation uses images captured at fixed 15 or 30 FPS, the inter-frame displacement is larger. Even under these challenging conditions, GSFlow achieves rendering quality comparable to 3DGS in terms of lighting, edges, and fine details, while significantly improving rendering speed. Additional visual comparisons under Vulkan implementation using a single RTX 2060 GPU are provided in Fig. 8, further validating the robustness of our method.